

- Dispensa — Progetto 0 Principiante
 - Convertitore di Unità di Misura
 - Indice
 - 1. Funzioni: def, parametri e return
 - Sintassi base
 - Esempio pratico (conversione)
 - Punti chiave
 - Funzioni senza return
 - 2. Condizionali: if / elif / else
 - Sintassi
 - Regole importanti
 - Esempio: menu di scelta
 - Operatori di confronto
 - Nota: == vs =
 - 3. Input e output: input(), print(), f-string
 - input() — leggere dati dall'utente
 - print() — stampare a schermo
 - f-string — formattazione moderna
 - Formattazione numerica con f-string
 - Allineamento nelle f-string
 - 4. Operazioni matematiche
 - Precedenza degli operatori
 - Applicazione al progetto
 - 5. Il ciclo while
 - Sintassi
 - Esempio: conto alla rovescia
 - while True con break
 - Questo è il pattern usato nel progetto!
 - 6. Conversione di tipo: float(), int()
 - float() — converte in numero decimale
 - int() — converte in numero intero
 - Errori comuni
 - Nel progetto
 - 7. Gestione errori base: try / except
 - Sintassi
 - Come funziona
 - Esempio completo per il progetto

- [None e il confronto is None](#)
- 8. Riepilogo e consigli
 - [Mappa concetti → dove li usi nel progetto](#)
 - [Suggerimenti pratici](#)
 - [Ordine consigliato di implementazione](#)
 - [Riferimenti ufficiali](#)

Dispensa — Progetto 0 Principiante

Convertitore di Unità di Misura

Obiettivo della dispensa: fornirti gli strumenti necessari per completare il progetto. In alcuni casi faremo riferimenti espliciti al codice che verrà usato direttamente nella soluzione.

Indice

1. Funzioni: `def`, parametri e `return`
 2. Condizionali: `if` / `elif` / `else`
 3. Input e output: `input()`, `print()`, f-string
 4. Operazioni matematiche
 5. Il ciclo `while`
 6. Conversione di tipo: `float()`, `int()`
 7. Gestione errori base: `try` / `except`
 8. Riepilogo e consigli
-

1. Funzioni: `def`, parametri e `return`

Una **funzione** è un blocco di codice riutilizzabile a cui diamo un nome. Serve a organizzare il programma in pezzi piccoli e leggibili.

Sintassi base

```
def nome_funzione(parametro1, parametro2):  
    """Docstring: spiega cosa fa la funzione."""  
    # corpo della funzione  
    risultato = parametro1 + parametro2  
    return risultato
```

- **def** — parola chiave che introduce la definizione di una funzione.
- **Nome** — segue le stesse regole delle variabili (**snake_case** è la convenzione).
- **Parametri** — variabili locali che ricevono i valori passati alla chiamata.
- **return** — restituisce un valore al chiamante. Se manca, la funzione restituisce **None**.

Esempio pratico (conversione)

```
def metri_a_piedi(metri):  
    """Converte metri in piedi. 1 metro = 3.28084 piedi."""  
    return metri * 3.28084  
  
# Chiamata alla funzione  
risultato = metri_a_piedi(10)  
print(risultato) # 32.8084
```

Punti chiave

Concetto	Descrizione
Parametro	La variabile nella <i>definizione</i> : <code>def f(x) → x</code> è il parametro
Argomento	Il valore nella <i>chiamata</i> : <code>f(10) → 10</code> è l'argomento
return	Termina la funzione e restituisce il valore
Docstring	Stringa tra triple virgolette subito dopo def — documentazione

Funzioni senza **return**

Se una funzione non ha **return**, restituisce **None**:

```
def saluta(nome):  
    print(f"Ciao {nome}!") # stampa, ma non restituisce nulla  
  
risultato = saluta("Mario")  
print(risultato) # None
```

Riferimento: [Defining Functions](#)

2. Condizionali: **if** / **elif** / **else**

Le condizioni permettono al programma di eseguire codice diverso in base a una condizione.

Sintassi

```
if condizione_1:  
    # eseguito se condizione_1 è True  
elif condizione_2:  
    # eseguito se condizione_1 è False E condizione_2 è True  
else:  
    # eseguito se TUTTE le condizioni precedenti sono False
```

Regole importanti

- **Indentazione obbligatoria:** Python usa l'indentazione (4 spazi) per delimitare i blocchi.
- **elif** è opzionale e può apparire più volte.
- **else** è opzionale e può apparire al massimo una volta (alla fine).
- Le condizioni sono valutate **dall'alto verso il basso**: la prima che è **True** "vince".

Esempio: menu di scelta

```
scelta = input("Scelta: ")  
  
if scelta == "1":
```

```
print("Hai scelto 1")
elif scelta == "2":
    print("Hai scelto 2")
elif scelta == "3":
    print("Hai scelto 3")
else:
    print("Scelta non valida!")
```

Operatori di confronto

Operatore	Significato	Esempio
<code>==</code>	Uguale a	<code>5 == 5</code> → <code>True</code>
<code>!=</code>	Diverso da	<code>5 != 3</code> → <code>True</code>
<code><</code>	Minore di	<code>3 < 5</code> → <code>True</code>
<code>></code>	Maggiore di	<code>5 > 3</code> → <code>True</code>
<code><=</code>	Minore o uguale	<code>5 <= 5</code> → <code>True</code>
<code>>=</code>	Maggiore o uguale	<code>5 >= 3</code> → <code>True</code>

Nota: `==` vs `=`

- `=` è l'**assegnamento**: `x = 5` (mette 5 dentro x)
- `==` è il **confronto**: `x == 5` (chiede "x è uguale a 5?")

3. Input e output: `input()`, `print()`, f-string

`input()` — leggere dati dall'utente

```
nome = input("Come ti chiami? ")
```

- Mostra il messaggio tra parentesi come prompt.
- **Restituisce SEMPRE una stringa** (anche se l'utente digita un numero!).

- Per ottenere un numero: `int(input(...))` o `float(input(...))`.

`print()` — stampare a schermo

```
print("Ciao mondo!")
print("Nome:", nome, "Età:", 25)
print() # riga vuota
```

- Accetta più argomenti separati da virgola (li separa con uno spazio).
- `print()` senza argomenti stampa una riga vuota.

f-string — formattazione moderna

Le **f-string** (formatted string literals) sono il modo più comodo per inserire variabili dentro una stringa. Si riconoscono dalla **f** prima delle virgolette.

```
nome = "Mario"
eta = 25
print(f"Mi chiamo {nome} e ho {eta} anni.")
# Output: Mi chiamo Mario e ho 25 anni.
```

Formattazione numerica con f-string

Per il progetto, è fondamentale formattare i numeri con un certo numero di decimali:

```
valore = 3.14159

# Mostra 2 decimali
print(f"{valore:.2f}")      # "3.14"

# Mostra 4 decimali
print(f"{valore:.4f}")      # "3.1416" (arrotondato!)

# Nessun decimale
print(f"{valore:.0f}")      # "3"

# Dentro una frase
print(f"Il valore è {valore:.2f} euro") # "Il valore è 3.14 euro"
```

La sintassi `:.Nf` significa:

- `:` → inizia la specifica di formato
- `.N` → N cifre dopo il punto decimale
- `f` → formato "fixed-point" (decimale fisso)

Allineamento nelle f-string

```
testo = "ciao"
print(f"{testo:<10}") # "ciao      " (allineato a sinistra, 10 caratteri)
print(f"{testo:>10}") # "          ciao" (allineato a destra)
print(f"{testo:^10}") # "   ciao   " (centrato)
```

Riferimento: [Formatted String Literals](#)

4. Operazioni matematiche

Python supporta tutte le operazioni aritmetiche di base:

Operatore	Operazione	Esempio	Risultato
<code>+</code>	Somma	<code>5 + 3</code>	<code>8</code>
<code>-</code>	Sottrazione	<code>5 - 3</code>	<code>2</code>
<code>*</code>	Moltiplicazione	<code>5 * 3</code>	<code>15</code>
<code>/</code>	Divisione (float)	<code>7 / 2</code>	<code>3.5</code>
<code>//</code>	Divisione intera	<code>7 // 2</code>	<code>3</code>
<code>%</code>	Modulo (resto)	<code>7 % 2</code>	<code>1</code>
<code>**</code>	Potenza	<code>2 ** 3</code>	<code>8</code>

Precedenza degli operatori

Come in matematica: `**` > `*`, `/`, `//`, `%` > `+`, `-`

```
risultato = 2 + 3 * 4    # 14, NON 20! (prima * poi +)
risultato = (2 + 3) * 4  # 20 (le parentesi cambiano l'ordine)
```

Applicazione al progetto

Le formule di conversione usano moltiplicazione e divisione:

```
# Celsius -> Fahrenheit: F = (C * 9/5) + 32
def celsius_a_fahrenheit(celsius):
    return (celsius * 9 / 5) + 32

# Fahrenheit -> Celsius: C = (F - 32) * 5/9
def fahrenheit_a_celsius(fahrenheit):
    return (fahrenheit - 32) * 5 / 9
```

Nota: $9/5$ in Python 3 dà 1.8 (divisione float), non 1 come in alcuni linguaggi.

5. Il ciclo `while`

Il ciclo `while` ripete un blocco di codice **finché la condizione è vera**.

Sintassi

```
while condizione:
    # questo blocco si ripete finché condizione è True
    # qualcosa deve cambiare per evitare un ciclo infinito!
```

Esempio: conto alla rovescia

```
contatore = 5
while contatore > 0:
    print(contatore)
    contatore -= 1    # equivale a: contatore = contatore - 1
print("PARTENZA!")
```


Output: 5, 4, 3, 2, 1, PARTENZA!

while True con break

Un pattern molto comune è il **ciclo infinito con uscita condizionata**:

```
while True:                # ciclo infinito
    scelta = input("Scelta (0 per uscire): ")
    if scelta == "0":
        break              # esce dal ciclo!
    print(f"Hai scelto: {scelta}")

print("Fine!") # questa riga viene eseguita dopo il break
```

- **while True** → il ciclo non si ferma MAI da solo.
- **break** → interrompe il ciclo immediatamente e il programma continua dopo il **while**.

Questo è il pattern usato nel progetto!

```
def main():
    while True:
        mostra_menu()
        scelta = input("Scelta: ")

        if scelta == "0":
            print("Arrivederci!")
            break
        elif scelta == "1":
            gestisci_lunghezza()
        elif scelta == "2":
            gestisci_peso()
        # ... ecc.
```

Riferimento: [The while statement](#)

6. Conversione di tipo: **float()**, **int()**

In Python, `input()` restituisce **sempre una stringa**. Per fare calcoli matematici con il valore inserito dall'utente, dobbiamo **convertirlo** in un numero.

`float()` — converte in numero decimale

```
testo = "3.14"
numero = float(testo)    # 3.14 (tipo float)
print(numero * 2)        # 6.28
```

`int()` — converte in numero intero

```
testo = "42"
numero = int(testo)      # 42 (tipo int)
```

Errori comuni

```
float("abc")           # ValueError! "abc" non è un numero
float("")               # ValueError! stringa vuota
int("3.14")             # ValueError! int() non accetta il punto decimale
float("3.14")           # OK -> 3.14
int(3.14)               # OK -> 3 (tronca il decimale)
```

Nel progetto

```
valore = float(input("Inserisci il valore in Metri: "))
```

Ma se l'utente scrive "abc", il programma **crasha** con `ValueError`. Per risolvere, usiamo `try/except` (sezione successiva).

Riferimento: `input()`

7. Gestione errori base: `try` / `except`

Quando un'operazione potrebbe fallire (es: conversione di tipo), possiamo **catturare l'errore** invece di far crashare il programma.

Sintassi

```
try:
    # codice che POTREBBE causare un errore
    numero = float(input("Numero: "))
except ValueError:
    # questo blocco viene eseguito SOLO se c'è un ValueError
    print("Non è un numero valido!")
```

Come funziona

1. Python esegue il codice dentro `try`.
2. Se **non ci sono errori** → salta `except` e continua normalmente.
3. Se **c'è un errore del tipo specificato** → esegue il blocco `except`.
4. Se **c'è un errore di tipo diverso** → il programma crasha comunque.

Esempio completo per il progetto

```
def chiedi_valore(messaggio):
    """Chiede un numero all'utente. Restituisce None se non valido."""
    try:
        valore = float(input(messaggio))
        return valore
    except ValueError:
        print("Devi inserire un numero valido.")
        return None
```

Uso:

```
valore = chiedi_valore("Inserisci Metri: ")
if valore is not None:
```

```
risultato = metri_a_piedi(valore)
print(f"Risultato: {risultato:.2f}")
```

None e il confronto `is None`

- `None` è un valore speciale che significa "nessun valore".
- Si confronta con `is`, non con `==`:
 - `if valore is None:` → **corretto**
 - `if valore is not None:` → **corretto**

Riferimento: [Handling Exceptions](#)

8. Riepilogo e consigli

Mappa concetti → dove li usi nel progetto

Concetto	Dove lo usi
<code>def + return</code>	Ogni funzione di conversione (<code>metri_a_piedi</code> , ecc.)
<code>if/elif/else</code>	Menu di scelta (categoria, tipo di conversione)
<code>input() + float()</code>	<code>chiedi_valore()</code> per leggere il numero dall'utente
<code>print() + f-string</code>	Tutti i messaggi e i risultati formattati
<code>:.2f</code>	Formattazione dei risultati con 2 decimali
<code>while True + break</code>	Ciclo principale del menu in <code>main()</code>
<code>try/except</code>	Gestione input non numerico in <code>chiedi_valore()</code>

Suggerimenti pratici

1. **Inizia dalle funzioni di conversione** — sono le più semplici (una riga ciascuna).
2. **Testa ogni funzione subito** — scrivi la funzione, poi `print(f(10))` per verificare.
3. **Il menu è solo un `while True` con `if/elif/else` dentro.**
4. **Non complicare le cose** — ogni funzione fa UNA sola cosa.

Ordine consigliato di implementazione

1. Scrivi le 10 funzioni di conversione (una riga ciascuna)
 2. Scrivi `mostra_menu()` (solo `print`)
 3. Scrivi `chiedi_valore()` (con `try/except`)
 4. Scrivi `main()` con il ciclo `while True`
 5. Testa tutto!
-

Riferimenti ufficiali

- [Funzioni](#)
- `input()`
- `print()`
- [f-string](#)
- `try/except`
- `while`